Decision Trees, Bagging, and SAMME Boosting on the Olivetti Faces Dataset: An Empirical Study of Robustness and Bias–Variance

Jahanvi Thumar
Roll No.: M26CSA016

Abstract

The assignment involves a comparison of decision trees, bagging, and SAMME AdaBoost when performing multi-class face classification with the Olivetti Faces dataset and using the raw flattened pixel values. The models are chosen on the basis of validation accuracy and are then evaluated on both clean and centrally masked test images. The bagging and boosting methods are implemented from scratch in order to examine the performance of ensembles, the level of prediction confidence, and the robustness towards missing visual information. To assess bias squared and variance, repeated resampling of the training set is employed, thus offering insights into model complexity, stability, and generalisation.

[1] [2] [3]

1. Introduction

The identification of faces is a significant task within the field of computer vision. In this assignment, decision trees, bagging, and multiclass SAMME AdaBoost are compared using the Olivetti Faces data set. The images are represented by their raw, flattened pixels without any dimensionality reduction, and the implementations of bagging and boosting are done from first principles.

The study looks at classification accuracy, the level of prediction confidence, and the bias–variance characteristics; it also investigates the model's robustness when facing a centrally masked area of the image in order to find out how ensemble methods influence stability and generalisation.

2. Dataset and Experimental Setup

We use the Olivetti Faces dataset and raw pixel features for all experiments. Table 1 summarizes the dataset, data split, and evaluation protocol. The split is generated once using the complete roll number as the random seed and is reused throughout Q1–Q3.

The masked test set is used only for final robustness evaluation; it is never used for training or model selection. In Q3, additional training sets are sampled with replacement, while the original test set remains fixed.

Table 1. Dataset and experimental setup.

Setting	Description
Dataset	Olivetti Faces
Images / identities	400 / 40
Image resolution	64×64
Features	4,096 raw pixel values
Train split	6 images per identity (240 total)
Validation split	2 images per identity (80 total)
Test split	2 images per identity (80 total)
Random seed	All roll-number digits, in order
Split generation	NumPy `default_rng`
Preprocessing	Flatten; no PCA or dimensionality reduction
Masking	Zero rows/columns 24–39 on test images
Model selection	Clean validation accuracy only

3. Q1: Decision tree baseline and bagging

We took a close look at a single decision tree using the Olivetti Faces dataset, which features 400 grayscale images representing 40 different identities. Each image, sized at 64×64, was flattened into 4,096 pixel features. We divided the data into 240 images for training, 80 for validation, and another 80 for testing. In total, we tested fifteen different configurations, playing around with five maximum-depth settings and three minimum leaf sizes. Ultimately, we chose the best configuration based solely on the clean validation accuracy.

3.1. Single decision tree

As we increase the depth of the decision tree, it starts to learn more detailed rules based on the raw pixel features. For instance, when we set the tree depth to 2, the training accuracy was only 7.50%. By the time we reached a depth of 16, that accuracy jumped to 42.50%. When we didn't limit depth, the training accuracy shot up to % with a minimum leaf size of one. For leaf sizes of two and four, the accuracy was still very high at 97.50%. Validation accuracy, which shows how well the model performs on new

Table 2. Performance of decision tree configurations.

Depth	Min. Leaf	Train (%)	Val. (%)
2	1	7.50	6.25
2	2	7.50	6.25
2	4	7.50	6.25
4	1	12.50	11.25
4	2	12.50	11.25
4	4	12.50	11.25
8	1	22.50	18.75
8	2	22.50	18.75
8	4	22.50	18.75
16	1	42.50	36.25
16	2	42.50	36.25
16	4	42.50	36.25
None	1	100.00	62.50
None	2	97.50	60.00
None	4	**97.50**	**65.00**

Table 3. Final single decision tree baseline results.

Metric	**Result**
Maximum depth	None
Minimum samples per leaf	4
Training accuracy (%)	97.50
Validation accuracy (%)	65.00
Clean test accuracy (%)	56.25
Number of leaves	40
Mean samples per leaf	6.00
Singleton leaf fraction	0.00

data, peaked at 65.00% with the chosen settings.

The decision tree achieved a training accuracy of 97.50%, but it only managed 56.25% accuracy on test data, showing a significant gap in how well it generalizes. With 40 leaves and about six training samples for each leaf, the model didn't have any leaves with a single training instance. Even though the minimum leaf-size rule kept those single samples out, the generalization gap remained.

3.2. Bagging from scratch

I used a method called bagging with 240 training images to help my model learn, sticking to the tree setup we discussed earlier. To get final predictions, combined the outputs from different numbers of trees—specifically 1, 5, 10, 25, 50, 100, and 200—using majority voting. I found that the best validation accuracy i got was 88.75% when using 50 trees, so I decided to go with that option. On average, each bootstrap sample contained about 152 distinct images, which is pretty close to what I expected.

Table 4. Bagging performance for different ensemble sizes.

B	Train (%)	Val. (%)	Test (%)	Distinct
1	72.50	42.50	45.00	155.00
5	93.75	57.50	58.75	152.20
10	98.33	76.25	71.25	151.70
25	100.00	81.25	76.25	151.24
50	100.00	88.75	90.00	151.04
100	100.00	87.50	91.25	151.68
200	100.00	87.50	91.25	152.08

Distinct denotes the mean number of distinct training samples.

Table 5. Theoretical bootstrap sample statistics for $n = 240$.

Metric	Value
Original training samples	240
Expected distinct samples	151.89
Expected fraction represented	63.29%

Table 6. Ensemble diagnostics for different numbers of trees.

B	OOB Acc. (%)	Disagreement	Vote Margin
1	35.29	–	1.0000
5	24.19	0.8238	0.2125
10	36.71	0.8072	0.2325
25	56.25	0.8047	0.2430
50	71.67	0.8027	0.2578
100	75.83	0.8039	0.2544
200	77.08	0.8059	0.2516

3.3. Ensemble diagnostics

The vote margin is defined as

$$m(x) = q_{(1)}(x) - q_{(2)}(x), \tag{1}$$

where $q_{(1)}(x)$ and $q_{(2)}(x)$ are the largest and second-largest class vote fractions.

The OOB accuracy generally increases with ensemble size, while pairwise disagreement remains nearly stable. The vote margin peaks at $B = 50$ and decreases slightly afterward, suggesting limited gains from adding more trees beyond this point.

3.4. Clean versus masked test images

The single decision tree managed to hit 56.25% accuracy with clean images and only 27.50% with masked ones. In contrast, bagging performed much better, achieving 90.00% accuracy on clean images and 73.75% on masked images. Interestingly, the tree had a higher mean confidence level

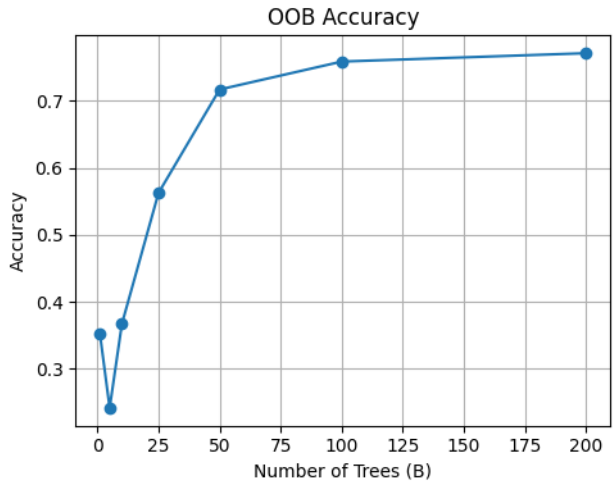


Figure 1. Bagging diagnostics versus ensemble size: (a) OOB accuracy

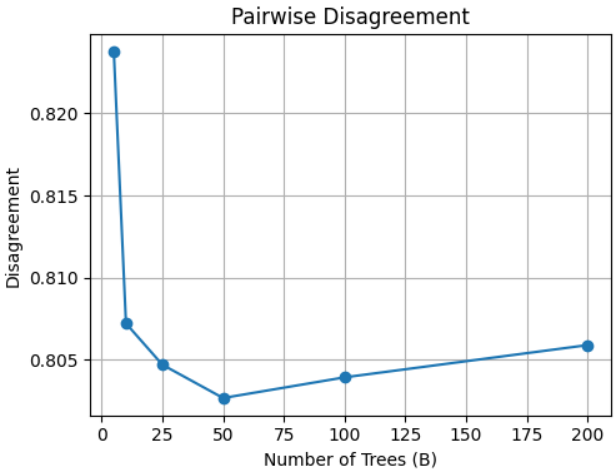


Figure 2. Bagging diagnostics versus ensemble size:(b) mean pairwise disagreement

Table 7. Test accuracy and mean prediction confidence.

	Tree		Bagging	
Test Set	Acc.	Conf.	Acc.	Conf.
Clean	56.25	0.982	90.00	0.391
Masked	27.50	1.000	73.75	0.292

(0.982 for clean images and 1.000 for masked) compared to bagging (0.391 for clean and 0.292 for masked), which shows that just because a model is confident doesn't mean it's always right.

Bagging helps to minimize errors by merging the results of trees that were trained on different bootstrap samples.

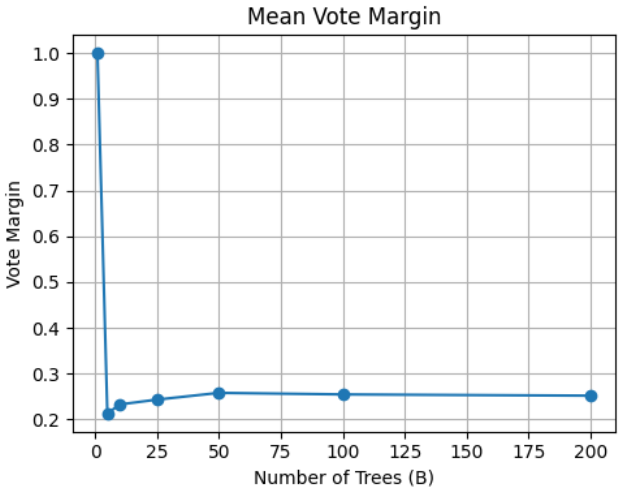


Figure 3. Bagging diagnostics versus ensemble size:(c) mean vote margin.

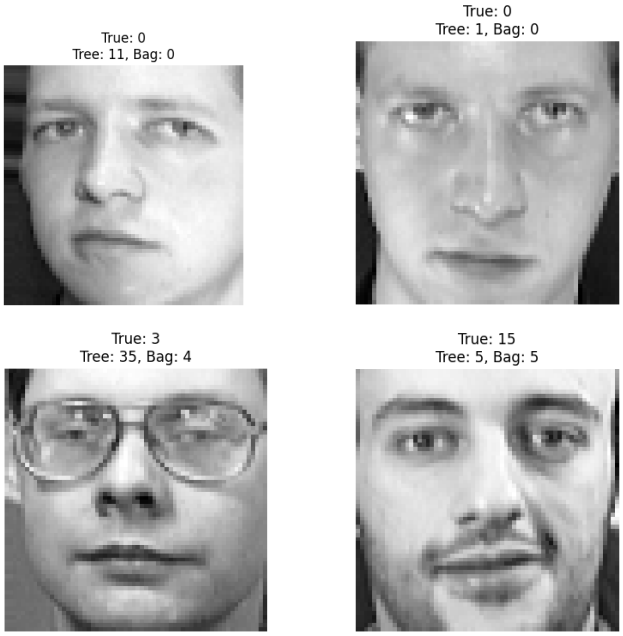


Figure 4. Examples of bagging correcting single-tree predictions (top) and cases where both models fail (bottom).

This way, the majority vote can help fix the mistakes made by individual trees. However, it struggles to correct errors effectively when the trees tend to make the same mistakes or when the images don't provide enough distinguishing features.

4. Q2: Multiclass boosting from scratch

4.1. One Complete SAMME Round

In this example, we explored a three-class scenario with six samples to illustrate a full round of SAMME boosting. At the start, every sample was given the same weight, which was $1/6$ for each one. We then compared the predictions made by the weak learner against the actual labels to determine the weighted classification error.

Table 8. Three-class example for one SAMME round.

Sample	True	Predicted	Incorrect	Initial Weight
1	0	0	No	0.1667
2	0	1	Yes	0.1667
3	1	1	No	0.1667
4	1	1	No	0.1667
5	2	0	Yes	0.1667
6	2	2	No	0.1667

$$\epsilon_t = \sum_{i=1}^{n} w_i \mathbf{1}[h_t(x_i) \neq y_i]. \tag{2}$$

$$\alpha_t = \log\left(\frac{1-\epsilon_t}{\epsilon_t}\right) + \log(K-1). \tag{3}$$

$$w_i^{(t+1)} = \frac{w_i^{(t)} \exp\left(\alpha_t \mathbf{1}[h_t(x_i) \neq y_i]\right)}{Z_t}, \tag{4}$$

Table 9. Normalized sample weights after one SAMME round.

Sample	Correct?	Initial Weight	Updated Weight
1	Yes	0.1667	0.0833
2	No	0.1667	0.3333
3	Yes	0.1667	0.0833
4	Yes	0.1667	0.0833
5	No	0.1667	0.3333
6	Yes	0.1667	0.0833
Total		1.0000	1.0000

In the case of multiclass SAMME, the threshold for random-guessing error is set at $1 - 1/K$. For our scenario, that means the threshold is $1 - 1/3 = 0.6667$. If a learner has an error rate ϵ_t of 0.6667 or higher, it gets rejected, and the boosting process will stop unless we can find a suitable learner. We also handle the situation of zero error separately to prevent division by zero: if a learner is perfect, it's accepted, and the boosting process comes to an end.

The weak learner managed to achieve a weighted error of 0.3333, which is below the random-guessing threshold of 0.6667. This resulted in a positive learner weight of 1.3863. After updating, the weights for the samples that were incorrectly classified rose to 0.3333, while the weights for those that were correctly classified dropped to 0.0833. The updated weights total one, marking the completion of a full SAMME round.

4.2. Hyperparameter selection and training curves

The 16 configurations were evaluated using clean validation accuracy. The configuration with the highest validation accuracy was selected, without using test-set performance for model selection.

The selected model used depth 3 and 100 boosting rounds, achieving 33.75% clean validation accuracy.

The training curves show how the weighted classification error, weak learner weights, and concentration of sample weights change across boosting rounds.

4.3. Boosting versus bagging

Performance comparison. Table 12 compares bagging and SAMME boosting for 25, 50, and 100 learners.

Bagging consistently outshines SAMME across all three ensemble sizes, hitting an impressive 90.00% clean test accuracy with both 50 and 100 learners. While SAMME does see some improvement at 100 rounds, it only manages to achieve a test accuracy of 38.75%. Interestingly, boosting requires less training time in these experiments, but bagging clearly delivers much stronger predictive performance.

Accuracy comparison. Figure 6 compares training, validation, and clean test accuracy for both methods.

High-weight training sample. Training face 56 has a true identity of 9 and a final sample weight of 0.03394, which accounts for about 3.39% of the normalized total weight. Its predictions fluctuate across the boosting rounds, and many of them don't align with its true identity. In the SAMME algorithm, misclassified samples get a boost in their relative weight, which prompts the next weak learners to pay more attention to these challenging cases. When a sample is correctly classified, its relative importance decreases after normalization, meaning its weight doesn't always have to go up in every round. Ultimately, the final weight is roughly 8.14 times its initial uniform weight of $1/240 \approx 0.00417$.

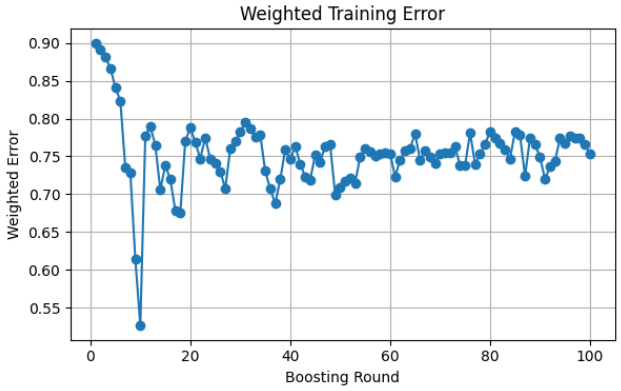
Figure 7 shows the weight history of this sample over the boosting rounds.

4.4. Masked test evaluation

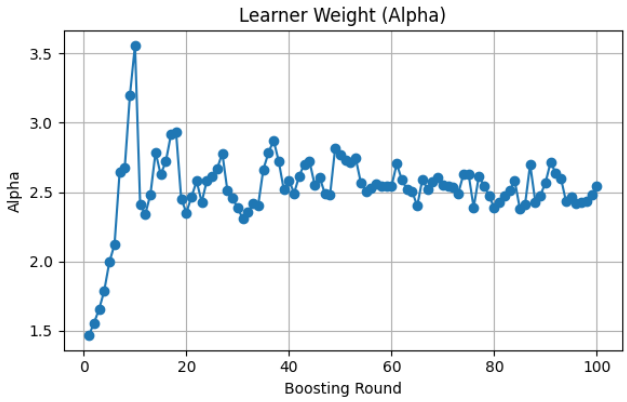
In the case of test face 4 (where the true identity is 2), we see some interesting shifts: masking alters bagging's prediction from 2 to 33, while SAMME's prediction changes from 4 to 20. Both models exhibit a significant level of disagreement, with scores of 0.929 and 0.906, which suggests there's quite

Table 10. SAMME hyperparameter selection using clean validation accuracy.

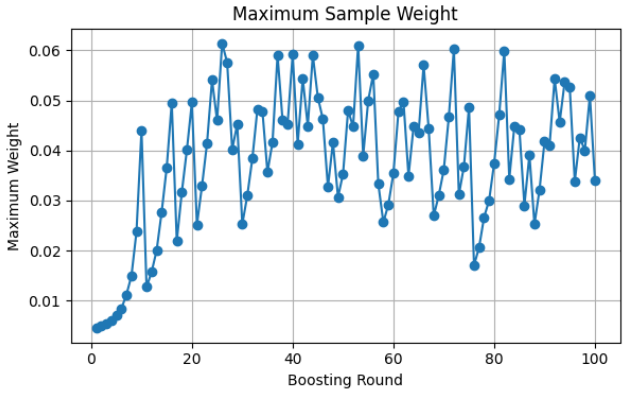
Depth	Requested Rounds	Actual Rounds	Train Accuracy (%)	Validation Accuracy (%)
1	10	10	5.00	5.00
1	25	25	5.00	3.75
1	50	50	13.33	6.25
1	100	100	15.83	10.00
2	10	10	12.50	5.00
2	25	25	17.50	7.50
2	50	50	31.67	10.00
2	100	100	38.33	13.75
3	10	10	27.50	13.75
3	25	25	47.50	20.00
3	50	50	47.92	17.50
3	100	100	95.00	**33.75**
4	10	10	24.58	10.00
4	25	25	75.00	27.50
4	50	50	55.83	18.75
4	100	100	78.75	27.50



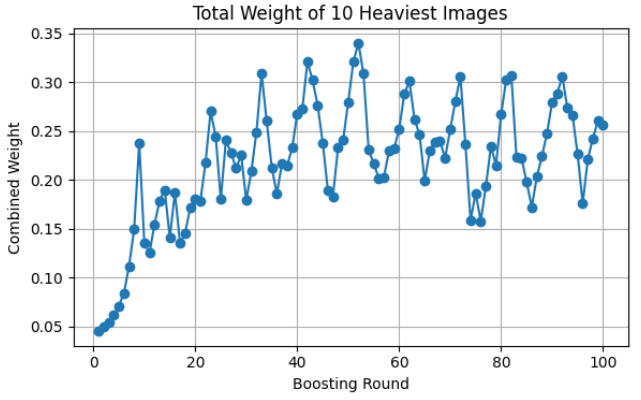
(a) Weighted training error



(b) Weak learner weight



(c) Maximum sample weight



(d) Total weight of the 10 heaviest samples

Figure 5. Training dynamics of the selected SAMME model with depth 3 and 100 rounds.

Table 11. Selected SAMME model.

Parameter	Value
Weak learner depth	3
Requested rounds	100
Actual accepted rounds	100
Validation accuracy	33.75%

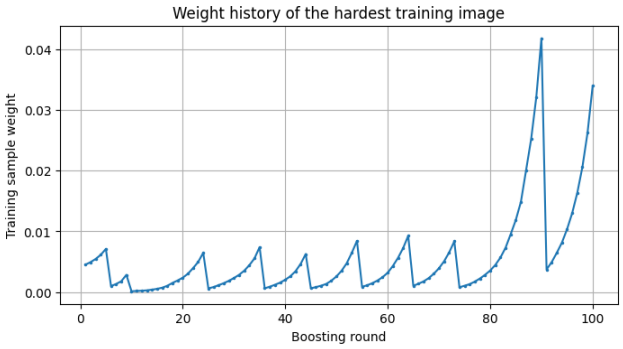


Figure 6. Training, validation, and clean test accuracy versus ensemble size for bagging and SAMME boosting.

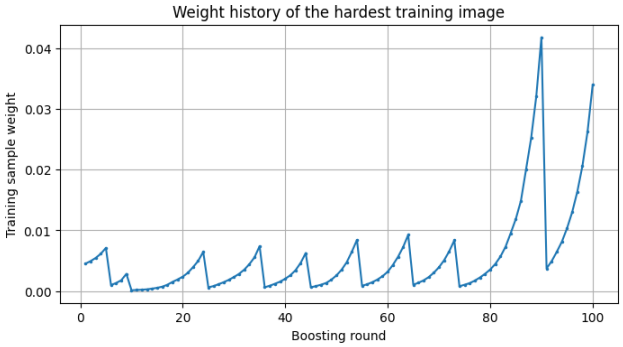


Figure 7. Sample-weight history of training face 56, whose true identity is 9, across SAMME boosting rounds.



Figure 8. Selected test face after masking the central 16×16 region. The true identity is 2.

a bit of uncertainty at play. Boosting tends to focus on the tougher samples, but that doesn't necessarily mean it will lead to accurate predictions when the images are masked. On the other hand, bagging seems to handle these masked images better, achieving an accuracy of 73.75%, compared to SAMME's 38.75%.

5. Q3: Bias and variance

5.1. Measure bias squared, variance and accuracy stability

We generated 30 stratified bootstrap resamples, sampling six training images with replacement per identity. The selected single tree, 50-tree bagging ensemble, and SAMME model were retrained without retuning and evaluated on the same 80 clean test images. Bias squared and variance were computed using class-probability vectors for the tree and normalized vote vectors for bagging and boosting.

As shown in Table 14, the single tree has the lowest bias squared (0.3745) but the highest variance (0.6355). Bagging reduces variance by 94.9% to 0.0322 and achieves the highest mean test accuracy (75.50%) and lowest accuracy standard deviation (0.0440). Boosting has the lowest variance (0.0150) but the highest bias squared (0.9149) and lowest accuracy (19.63%). Thus, low variance alone does not guarantee high accuracy; bagging provides the best overall performance in this experiment.

5.2. Study the effect of tree depth

We took a close look at single decision trees and 100-tree bagging ensembles, experimenting with shallow, selected, and deep tree depths across 30 bootstrap resamples. All models were assessed using the same clean test set, and we didn't make any further adjustments. You can find the details in Table 15, which shows the calculated bias squared, variance, and mean test accuracy.

To understand variance reduction, we looked at the difference between the variance of single trees and that of bagging at each depth. The findings shed light on how tree depth influences prediction stability and how well bagging manages to cut down variance. By examining the measured bias, variance, and accuracy, we can determine if deeper trees truly enhance predictive performance and whether bagging offers more advantages at specific depths.

$$\Delta V = V_{\text{tree}} - V_{\text{bag}}.$$

$$\text{Reduction (\%)} = \frac{V_{\text{tree}} - V_{\text{bag}}}{V_{\text{tree}}} \times 100.$$

5.3. Does the deepest tree benefit most from bagging?

Just because a tree is deeper doesn't mean it will give you the biggest boost in accuracy when using bagging. In our findings, the most significant improvement popped up at a depth of 20, where accuracy jumped from 31.46% to 66.71%—that's a whopping increase of 35.25 percentage

Table 12. Performance comparison of bagging and SAMME boosting.

Method	Learners	Train Acc. (%)	Val. Acc. (%)	Test Acc. (%)	Time (s)
Bagging	25	100.00	80.00	80.00	65.3940
SAMME	25	47.50	20.00	26.25	13.7971
Bagging	50	100.00	86.25	90.00	138.8948
SAMME	50	47.92	17.50	22.50	26.5988
Bagging	100	100.00	86.25	90.00	252.8261
SAMME	100	95.00	33.75	38.75	54.4637

Table 13. Clean and masked test performance.

Dataset	Model	Acc. (%)	Conf.
Clean	Single Tree	56.25	0.9815
Clean	Bagging (50)	90.00	0.3910
Clean	SAMME	38.75	0.1000
Masked	Single Tree	27.50	1.0000
Masked	Bagging (50)	73.75	0.2923
Masked	SAMME	38.75	0.0979

Table 14. Measured bias, variance, and test accuracy stability across 30 bootstrap repetitions.

Model	Bias^2	Variance	Mean Acc.	Std. Acc.
Single Tree	0.3745	0.6355	0.4950	0.0456
Bagging (50)	0.5242	0.0322	0.7550	0.0440
SAMME Boost.	0.9149	0.0150	0.1963	0.0499

Table 15. Effect of tree depth on bias, variance, and test accuracy across 30 resamples.

Model	Bias^2	Variance	Acc.
Tree (D=2)	0.9189	0.0363	0.0550
Bagging (D=2)	0.9319	0.0113	0.1463
Tree (D=All)	0.4063	0.6720	0.4608
Bagging (D=All)	0.5304	0.0258	0.7617
Tree (D=20)	0.5864	0.3210	0.3146
Bagging (D=20)	0.6261	0.0228	0.6671

Table 16. Variance reduction and accuracy improvement achieved by bagging at different depths.

Depth	Var. Red. (%)	ΔV	ΔAcc.
Shallow (2)	68.76	0.0249	0.0913
Selected (All)	96.17	0.6463	0.3008
Deep (20)	92.91	0.2983	0.3525

Table 17. Accuracy improvement and bias comparison by tree depth.

Depth	Single Acc.	Bag Acc.	ΔAcc.	Single Bias^2	Bag Bias^2
Shallow (2)	0.0550	0.1463	0.0913	0.9189	0.9319
Selected (All)	0.4608	0.7617	0.3008	0.4063	0.5304
Deep (20)	0.3146	0.6671	0.3525	0.5864	0.6261

points! In comparison, the unrestricted tree saw a 30.08-point boost, while the tree at depth 2 only improved by 9.13 points.

Table 18. Variance, variance reduction, leaf occupancy, and disagreement by tree depth.

Depth	Single Var.	Bag Var.	Var. Red. (%)	Leaf Occ.	Disagr.
Shallow (2)	0.0363	0.0113	68.76	80.0000	0.9621
Selected (All)	0.6720	0.0258	96.17	6.0000	0.8373
Deep (20)	0.3210	0.0228	92.91	10.9091	0.8764

Interestingly, the unrestricted tree had the highest variance at 0.6720, but bagging managed to slash that by an impressive 96.17%. At depth 20, variance dropped by 92.91%, and for the shallower tree, it decreased by 68.76%. These findings highlight that reducing variance and improving accuracy don't always align perfectly. In fact, we noticed that bias squared increased after bagging at all three depths, showing that cutting down on variance doesn't automatically mean bias will decrease.

When we look at leaf occupancy and singleton-leaf fractions, we can see how much predictions rely on specific training examples. Meanwhile, pairwise tree disagreement helps us understand the differences between trees trained on bootstrap samples. All these metrics, along with the improvements in accuracy and reductions in variance, shed light on why bagging works so well at different depths. So, it's clear that the optimal depth for bagging should be based on actual results rather than just the depth itself.

5.4. Final comparison and conclusion

Table 19. Final comparison of selected models on clean and masked test sets.

Model	Clean	Masked	Bias^2	Variance
Single Tree	0.5625	0.2750	0.3745	0.6355
Bagging (50)	0.9000	0.7375	0.5242	0.0322
SAMME Boost.	0.3875	0.3875	0.9149	0.0150

When it comes to model stability, SAMME boosting stands out with the lowest variance measured at 0.0150, closely followed by bagging at 0.0322. In contrast, the single tree model shows a much higher variance of 0.6355. This suggests that boosting offers the most consistent prediction vectors across different resamples, although bagging also does a great job of reducing variability.

Now, let's talk about sensitivity to masking. The single tree model is the most affected by missing data, with its accuracy plummeting from 56.25% to just 27.50%, a drop of 28.75 percentage points. Bagging sees a decrease from 90.00% to 73.75%, which is a drop of 16.25 points, while boosting holds steady at 38.75%. However, just because boosting's accuracy doesn't change doesn't mean its predictions are static; some faces might shift from being correctly identified to incorrectly identified, and vice versa, balancing out the overall accuracy.

Finally, let's consider variance versus accuracy. The model with the lowest variance isn't always the most accurate. Even though boosting has the least variance, it only achieves an accuracy of 38.75% on both test sets and has the highest squared bias at 0.9149. On the other hand, bagging excels with the highest accuracies on both clean and masked tests, scoring 90.00% and 73.75%, respectively. This demonstrates that bagging offers the best overall predictive performance and robustness in this experiment.

In summary, bagging strikes the best balance between accuracy, stability, and robustness, while boosting's low variance doesn't quite make up for its high bias and lackluster classification accuracy.

References

[1] Leo Breiman. Bagging predictors. *Machine Learning*, 24: 123–140, 1996. 1

[2] Leo Breiman et al. *Classification and Regression Trees*. Wadsworth, 1984. 1

[3] scikit-learn developers. Olivetti faces dataset. https://scikit-learn.org/stable/modules/generated/sklearn.datasets.fetch_olivetti_faces.html. 1